

CS 112

Introduction to Data Structures

WEEK 09

Stacks and Queues

Eric Araújo

Calvin University · Fall 2026

This Week

- 1 Abstract Data Types
- 2 The Stack ADT — LIFO
- 3 Implementing a stack
- 4 The Queue ADT — FIFO
- 5 Why the naive array queue fails
- 6 Circular arrays **KEY**
- 7 Deques

Abstract Data Type

An ADT is a promise about **behaviour**, with no commitment about **storage**.

The ADT says

- what operations exist
- what they mean
- what they cost

The ADT doesn't say

- array or linked list
- where the front is
- how it grows

Interface versus implementation — the same idea as week 3's public/private, now applied to a whole data structure.

The Stack: Last In, First Out

```
push(item)    // put on top
pop()         // take off top
peekTop()     // look, don't take
isEmpty()
getSize()
```

top →	white
green	
blue	

A stack of plates. Push green, push white, pop — you get **white** back, because it was last on.

Stacks Are Everywhere

- **Ctrl-Z** — the last change is the first undone
- **Browser back button** — the page you just left is on top
- **The run-time stack** — functions return in reverse call order
- **Matching brackets** — push an opener, pop when you meet its closer

Whenever "most recent first" is the right answer, a stack is the right structure. You'll meet the run-time stack again next week, in recursion.

What does this print?

```
s.push(7);  
s.push(8);  
s.push(9);  
while (!s.isEmpty()) {  
    cout << s.pop() << " ";  
}
```

A. 7 8 9

B. 9 8 7

C. 9 7 8

D. 7 9 8

Implementing a Stack

Array — top at the END

```
void push(const Item &it) {  
    if (isFull()) throw ...;  
    myArray[mySize] = it;  
    ++mySize;  
}
```

$O(1)$. Top at index 0 would shift everything — $O(n)$.

Linked list — top at the FRONT

```
void push(const Item &it) {  
    myList.prepend(it);  
}
```

$O(1)$. Top at the back would need a walk to find it — $O(n)$.

Same ADT, two implementations, one design rule: **put the action where the structure is fast.**

TALK TO YOUR NEIGHBOR · TTYN

Implementing a stack with a linked list, where should the top be?

- A.** At the front — prepend and remove-first are both $O(1)$
- B.** At the back — that's where append is
- C.** Either; both are $O(1)$
- D.** In the middle, to balance the cost

The Queue: First In, First Out

```
add(item)      // join the back
remove()       // serve the front
peekFront()
isEmpty()
getSize()
```

front 11 → 22 → 33 back

Serve at the front, join at the back — a supermarket queue.

Also called First Come, First Served. Print jobs, request handling, breadth-first search — all queues.

The Obvious Array Queue Is Wrong

Front at index 0, back at `mySize - 1`. Adding is easy. Removing?

before	[11 22 33 _]
remove()	[22 33 _ _] everything shifted down

Every `remove()` shifts every remaining item: **$O(n)$** . For a structure whose whole job is serving the front, that's a bad trade.

Don't Move the Data — Move the Ends

index	[0]	[1]	[2]	[3]	[4]
data	—	22	33	44	—
		head = 1		tail = 3	

Keep two indices. `remove()` just moves head forward — no shifting, **O(1)**.

...And Wrap Around the End

index	[0]	[1]	[2]	[3]	[4]
data	66	—	33	44	55
head = 2		tail = 0		(wrapped)	

```
tail = (tail + 1) % myCapacity;    // advance with wraparound  
head = (head + 1) % myCapacity;
```

The modulo turns the array into a **circle**. Both ends move; nothing is ever copied.

With `head == tail`, is the circular queue empty or full?

- A.** Empty
- B.** Full
- C.** Ambiguous — you can't tell without more information
- D.** Impossible — they can never be equal

Circular Array: The Trade

Wins

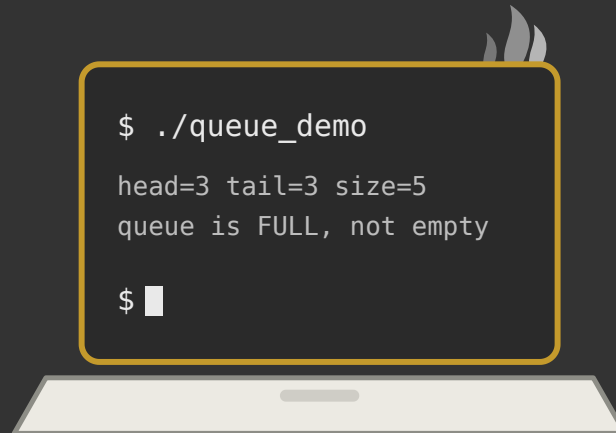
- Every operation $O(1)$
- No shifting, ever
- Almost no memory overhead

Costs

- Fiddly index arithmetic
- Full vs. empty needs care
- Fixed capacity unless you grow it

A linked-list queue avoids the fiddliness — add at the tail, remove at the head, both $O(1)$ — at the cost of a pointer per item and a new one on every insert.

Demo



Watching head and tail chase each other around.

(and the moment they collide)

Dequeues

Pronounced "deck" — a **d**ouble-**e**nded **q**ueue.

```
#include <deque>

deque<int> d;
d.push_front(1);    d.push_back(2);
d.pop_front();      d.pop_back();
```

Push and pop at either end, all $O(1)$. A deque can act as a stack *or* a queue — which is why the STL's stack and queue are built on one by default.

Cost Summary

Structure	Add	Remove	Peek
Stack (array, top at end)	$O(1)$	$O(1)$	$O(1)$
Stack (list, top at front)	$O(1)$	$O(1)$	$O(1)$
Queue (naive array)	$O(1)$	$O(n)$	$O(1)$
Queue (circular array)	$O(1)$	$O(1)$	$O(1)$
Queue (linked list)	$O(1)$	$O(1)$	$O(1)$

One row is worse than the others, and it's the one you'd write first without thinking. That's the lesson.

Week 09 Recap

- An **ADT** fixes the behaviour and leaves the storage open
- Stack = LIFO; Queue = FIFO
- Put the action where the structure is fast — end of an array, front of a list
- A naive array queue makes `remove()` $O(n)$; a **circular array** fixes it
- `(i + 1) % capacity` is the whole trick
- `head == tail` is ambiguous — keep a size, or leave a gap

Next: a09 →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**

